Reinforcement Learning and Symmetry in the Braid Group:

A MatrixNet Approach to Canonical Form Learning

Kevin Lee and Gabriel Wilkinson

March 2, 2026

Abstract

A core computational challenge in knot theory is the word problem for braids: deciding when two different sequences of crossings actually give you the same braid. In this paper, we lay out a theoretical blueprint for tackling this with reinforcement learning (RL) and representation theory. The idea is to have an RL agent navigate the space of braid words toward a unique canonical target—the Garside normal form. The main obstacle is that this space is huge and full of redundancy, since many different words can represent the same braid. To deal with this, we bring in MatrixNet, a neural architecture that learns invertible matrix representations respecting algebraic symmetry. Because MatrixNet maps equivalent words to approximately the same matrix, an RL agent can skip redundant exploration entirely.

We also outline a proposed extension of the MatrixNet framework to predict the vanishing of the Plamenevskaya transverse invariant $\psi(\beta)$ in Khovanov homology. By framing this #P-hard computation as binary classification on braid words, we argue that MatrixNet's architecture is well-suited to the task and describe how such an experiment could be set up. We wrap up by discussing potential applications to topological robotics, multi-robot coordination, and quantum computation.

Contents

1 Introduction **4**
1.1 Motivation: The Word Problem for Braids 4
1.2 The Central Goal of This Paper 4
1.3 Why These Three Ideas Belong Together 5

2 Reinforcement Learning **5**

3 Group Symmetry and Representation Theory **7**
3.1 Groups, Generators, and Presentations 7
3.2 Group Representations 8
3.3 Group Actions and Orbits 8

4 Knot Theory and the Braid Group **8**
4.1 Knots and Why They Are Hard to Classify 8
4.2 The Braid Group B_n 9
4.3 Alexander's Theorem: Braids Encode Knots 9
4.4 The Garside Normal Form 9
4.5 The Word Problem as an RL Environment 10

5 MatrixNet: Learning over Symmetry Groups **10**
5.1 Overview: From Fixed to Learned Representations 10
5.2 Jordan–Hölder Multiplicities: The Validation Task 11
5.3 Architecture 11
5.4 The Relation Loss 11
5.5 Empirical Results on B_3 12

6 Reinforcement Learning via MatrixNet on the Braid Group **12**
6.1 Assembling the Framework 12
6.2 The State Representation 13
6.3 The Action Space 13
6.4 The Reward Function 13
6.5 Policy Learning 13

7 Further Applications **14**

8 Proposed Extension: Predicting the Plamenevskaya Transverse Invariant **14**
8.1 Contact Geometry and Transverse Knots 14

8.2 Khovanov Homology 15
8.3 The Plamenevskaya Invariant $\psi(\beta)$ 15
8.3.1 ψ as a Quasipositivity Obstruction 15
8.4 Proposed Task: Binary Classification of $\psi(\beta)$ 16
8.5 Why MatrixNet's Architecture Fits This Problem 16
8.6 How We Would Set This Up 17
8.6.1 Dataset Construction 17
8.6.2 Loss Function 17
8.6.3 What Success Would Look Like 17

9 Conclusion **18**

1 Introduction

1.1 Motivation: The Word Problem for Braids

Take a piece of string, tie a knot in it, and glue the ends together. You get a closed loop. Two loops count as "the same knot" if you can wiggle one into the other without cutting anything. Deciding when two loops are actually equivalent is one of the oldest open problems in topology—and it turns out to have a surprisingly clean algebraic reformulation through *braid groups*.

A braid is n strands running from a top row of points down to a bottom row, where strands are allowed to cross over each other but always move downward. Any knot can be built by taking a braid and connecting the top endpoints to the bottom ones (Alexander's theorem, Section 4.3). Algebraically, braids are described as *words* in *generators* $\sigma_1, \ldots, \sigma_{n-1}$, where σ_i means "strand i crosses over strand $i+1$" (see Definition 4.2).

The problem is that lots of different words give you the same braid—they describe the same tangle, just written differently. Asking when two words are equivalent is the *word problem* for braid groups. Garside solved this in 1969 by defining a canonical form: a unique "normal" way to write every braid (Section 4.4). Two words are equivalent iff they reduce to the same Garside normal form.

1.2 The Central Goal of This Paper

Getting to the Garside normal form is a sequential process—you apply algebraic moves one at a time until you reach the canonical representative. This is a natural fit for *reinforcement learning*: an agent starts with some braid word, applies moves step by step, and gets rewarded for making progress toward normal form. The difficulty is that the state space is enormous and highly redundant (many words = same braid), so a naïve agent wastes time exploring equivalent states.

This is where *MatrixNet* [1] comes in. It learns to assign each generator an invertible matrix, then represents a braid word as the product of those matrices. Equivalent words produce (approximately) the same product, so the RL agent automatically treats them the same.

Beyond the RL framework, we also propose that MatrixNet could be applied to predict the vanishing of the *Plamenevskaya transverse invariant* $\psi(\beta)$ in Khovanov homology directly from braid words. We haven't implemented this yet, but we lay out why MatrixNet's architecture is a good fit and how such an experiment would work.

Central Thesis. We propose a theoretical blueprint in which an RL agent, equipped with MatrixNet's symmetry-aware representations, navigates braid words in B_n toward the Garside normal form, effectively solving the word problem through sequential decision-making. We additionally outline how these same representations could be applied to predict the vanishing of $\psi(\beta)$ in Khovanov homology, and argue that MatrixNet's algebraic inductive biases make it well-suited to capturing this kind of topological information.

1.3 Why These Three Ideas Belong Together

The three pillars—RL, braid groups, and MatrixNet—each fill a specific gap:

- **Braid groups** (Section 4) give us the mathematical playground: a discrete, combinatorial space with exact algebraic symmetry and a well-defined canonical target.
- **Reinforcement learning** (Section 2) gives us the right optimization paradigm: step-by-step decision-making with rewards, which fits braid normalization naturally.
- **MatrixNet** (Section 5) gives us the representation: a learned, symmetry-respecting embedding that keeps the RL agent from wasting time on equivalent states.

Figure 1 on the next page shows how these pieces connect. It's worth coming back to as each component is introduced.

Paper organization. Section 2 covers reinforcement learning basics. Section 3 develops the group theory and representation theory we need. Section 4 introduces the braid group and the Garside normal form. Section 5 describes MatrixNet, including the Jordan–Hölder validation task. Section 6 assembles the full RL pipeline. Section 7 discusses further applications. Section 8 outlines our proposed extension to the Plamenevskaya invariant. Section 9 concludes.

With the big picture in place, we now build up the background, starting with the learning paradigm itself.

2 Reinforcement Learning

In reinforcement learning (RL) [5], an *agent* interacts with an *environment* over discrete time steps. At step t, the agent sees a state $s_t \in \mathcal{S}$, picks an action $a_t \in \mathcal{A}$ according to its *policy* $\pi(a \mid s)$, gets a scalar reward $r_t = R(s_t, a_t)$, and the environment moves to s_{t+1}. The objective is to find a policy maximizing $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$, the expected cumulative discounted return ($\gamma \in [0, 1)$).

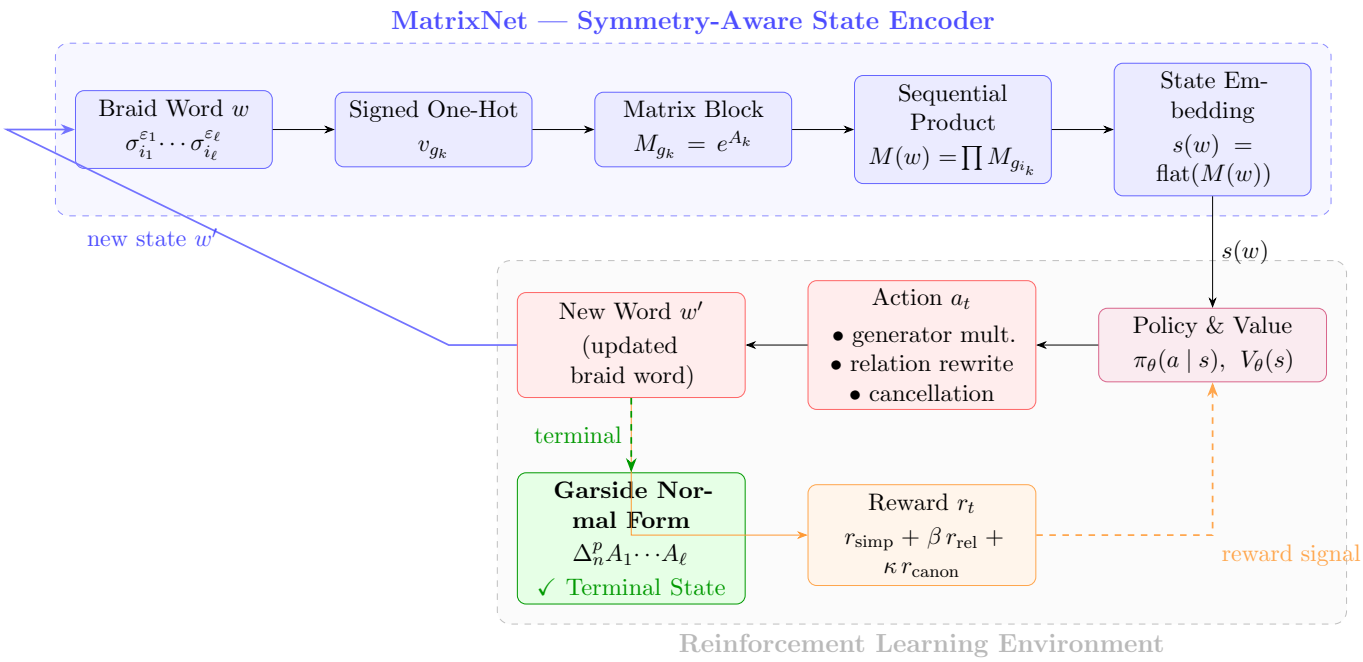

Figure 1: Complete framework overview. **Top (blue) — MatrixNet:** The braid word w gets a signed one-hot encoding, passes through learned matrix blocks ($M_{g_k} = e^{A_k}$), and is multiplied sequentially to produce the state embedding $s(w)$. Equivalent words yield the same $M(w)$, so the RL agent never re-explores redundant states. **Bottom (gray) — RL environment:** The policy picks an action a_t; the environment applies it to get w'; a reward r_t comes back; and the new word is re-encoded by MatrixNet. The blue back-loop shows how w' feeds back into MatrixNet. The episode ends (dashed green) when w' reaches the Garside normal form $\Delta_n^p A_1 \cdots A_\ell$ (Section 4.4).

The key difference from supervised learning is that nobody tells the agent the right answer—it has to figure out good behavior through trial and error. Actions have delayed consequences (*temporal credit assignment*), and there's a constant tension between trying new things (*exploration*) and sticking with what works (*exploitation*). Modern deep RL either approximates the optimal value function $Q^*(s, a)$ (e.g., DQN [6]) or directly optimizes a parameterized policy π_θ via policy gradients. Actor–critic methods do both.

Key Idea

Garside normalization is sequential, has no single "correct" action at each step, and terminates at a well-defined goal state. This is exactly the kind of problem RL is good at—and where supervised learning struggles, since we don't have a labeled dataset of optimal move sequences.

The algebraic symmetry of the braid group—the fact that many words represent the same element—is both a challenge and an opportunity. A naïve agent treats each word as its own state; a symmetry-aware agent explores much more efficiently. This is the motivation for MatrixNet (Section 5).

Before we can define the braid group, we need some algebra. Section 3 covers groups and their matrix representations.

3 Group Symmetry and Representation Theory

3.1 Groups, Generators, and Presentations

A *group* $(G, \circ)$ is a set equipped with an associative binary operation, an identity element e, and inverses for every element. If the operation commutes, the group is *abelian*. Some standard examples: $(\mathbb{Z}, +)$, the invertible matrices $\mathrm{GL}(n, \mathbb{R})$, and the symmetric group S_n. The braid group B_n (Section 4.2) is infinite and non-abelian.

A *generating set* $S = \{g_1, \ldots, g_k\}$ is a subset whose elements can be multiplied (with inverses) to produce every element of the group. A product $h = g_{i_1}^{\varepsilon_1} \cdots g_{i_\ell}^{\varepsilon_\ell}$ is called a *word*. *Relations* are equations among words that hold in G, and the pair $\langle S \mid R \rangle$ is a *presentation* of G. The *free group* $F_S = \langle S \mid \emptyset \rangle$ has no relations beyond the group axioms. The braid group comes from F_S by imposing two specific families of relations.

3.2 Group Representations

A *(matrix) representation* of G is a homomorphism $\rho : G \to \mathrm{GL}(d, \mathbb{R})$ satisfying $\rho(g \circ h) = \rho(g) \cdot \rho(h)$. For a word $w = g_{i_1} \cdots g_{i_\ell}$:

$$\rho(w) = \rho(g_{i_1}) \cdots \rho(g_{i_\ell}). \tag{1}$$

This is the property MatrixNet exploits: the matrix for a word is just the product of the matrices for its generators. If $w_1 \equiv w_2$ in G, then $\rho(w_1) = \rho(w_2)$—the representation is *group relation invariant*.

To measure how well a learned representation satisfies the relations, we use the *Frobenius norm* $\|A\|_F = \sqrt{\sum_{ij} a_{ij}^2}$, penalizing via $\|M(r) - I_d\|_F^2$ (zero when a relation is exactly satisfied).

3.3 Group Actions and Orbits

A group G *acts* on a set X via a map $\cdot : G \times X \to X$ with $e \cdot x = x$ and $(gh) \cdot x = g \cdot (h \cdot x)$. The *orbit* $\mathrm{Orb}(x) = \{g \cdot x : g \in G\}$ collects all equivalent elements. In the braid group, two words share an orbit iff they represent the same element. An RL agent whose representation is constant on orbits doesn't need to explore each word separately—it gets a *symmetry-equivariant policy* $\pi(a \mid s) = \pi(a \mid g \cdot s)$ for free.

4 Knot Theory and the Braid Group

4.1 Knots and Why They Are Hard to Classify

Definition 4.1. A *knot* is a smooth closed curve in $\mathbb{R}^3$ with no loose ends (formally, a smooth embedding $S^1 \hookrightarrow S^3$). Two knots are *equivalent* if one can be continuously deformed into the other without cutting or self-intersection.

The simplest knot is the *unknot*: just a plain circle. Even deciding whether a given knot is the unknot is computationally hard in general. Knot invariants—things like the crossing number or the Jones polynomial [7]—can sometimes distinguish knots, but no one has found a complete and efficient invariant.

4.2 The Braid Group B_n

Emil Artin introduced the braid group in 1925 [2]. Geometrically, a braid on n strands is n strands running from a top row of n points to a bottom row, always moving downward. Two braids are equivalent if you can deform one into the other while keeping the endpoints fixed.

Definition 4.2. The *braid group on n strands*, B_n, has the Artin presentation:

$$B_n = \left\langle \sigma_1, \ldots, \sigma_{n-1} \;\middle|\; \begin{array}{ll} \sigma_i\sigma_j = \sigma_j\sigma_i & \text{if } |i-j| > 1, \\ \sigma_i\sigma_{i+1}\sigma_i = \sigma_{i+1}\sigma_i\sigma_{i+1} & \text{for all valid } i \end{array} \right\rangle. \tag{2}$$

Here σ_i represents strand i crossing *over* strand $i+1$, and σ_i^{-1} is the reverse crossing. Every element of B_n is a word $w = \sigma_{i_1}^{\varepsilon_1} \cdots \sigma_{i_\ell}^{\varepsilon_\ell}$ with $\varepsilon_j \in \{+1, -1\}$.

The two relations have nice geometric interpretations:

- **Far commutativity:** $\sigma_i\sigma_j = \sigma_j\sigma_i$ when $|i-j| > 1$. Distant crossings don't interact, so the order doesn't matter.
- **Braid relation:** $\sigma_i\sigma_{i+1}\sigma_i = \sigma_{i+1}\sigma_i\sigma_{i+1}$. This is the Reidemeister III move: two ways of passing three adjacent strands through each other that give the same result.

Because of these relations, many different words can represent the same braid element. For instance, $\sigma_1\sigma_2\sigma_1 = \sigma_2\sigma_1\sigma_2$ in B_3. The *word problem* is: given two words u and v, do they represent the same element?

4.3 Alexander's Theorem: Braids Encode Knots

Theorem 4.3 (Alexander, 1923 [4])**.** *Every knot or link in S^3 is ambient isotopic to the closure of some braid.*

You form the closure of a braid $w \in B_n$ by connecting each top endpoint to the corresponding bottom endpoint. Alexander's theorem says every knot arises this way, so algorithms for the braid word problem have direct implications for knot equivalence.

4.4 The Garside Normal Form

Garside (1969) solved the word problem by showing every braid has a unique canonical form [3]. The key ingredient is the *Garside element* $\Delta_n = (\sigma_1)(\sigma_2\sigma_1)\cdots(\sigma_{n-1}\cdots\sigma_1)$, which is a half-twist of all n strands.

Theorem 4.4 (Garside, 1969 [3]). *Every element $w \in B_n$ has a unique representation*

$$w = \Delta_n^p \cdot A_1 \cdot A_2 \cdots A_\ell, \tag{3}$$

where $p \in \mathbb{Z}$ and $A_1, \ldots, A_\ell$ are positive simple elements satisfying a left-greedy condition. This is the Garside normal form.

Two braid words are equivalent iff they have the same Garside normal form. This gives us both a decision procedure for the word problem and the target for our RL agent.

4.5 The Word Problem as an RL Environment

Now we have everything we need to set up Garside normalization as an RL problem:

- **State:** a braid word $w \in B_n$.
- **Actions:** multiply by $\sigma_i^{\pm 1}$ on the left or right; rewrite a subword using a braid relation; cancel adjacent inverse pairs $\sigma_i \sigma_i^{-1} = e$.
- **Reward:** positive for shortening the word or reaching normal form; negative for lengthening or relation violations.
- **Terminal state:** the Garside normal form of w.

The main challenge is the massive, redundant state space. A naïve RL agent treats every word as a different state and explores uselessly. The fix: a symmetry-aware state representation, which is exactly what MatrixNet provides.

5 MatrixNet: Learning over Symmetry Groups

5.1 Overview: From Fixed to Learned Representations

MatrixNet [1] addresses a simple question: given a group $G = \langle S \mid R \rangle$ and a task $f : G \to \mathbb{R}^c$, how should we encode group elements for a neural network so that the encoding doesn't depend on which word we happened to pick?

The textbook approach uses fixed representations from the known representation theory of G. That works when you know the representation theory, but the braid group B_n is infinite and its representations haven't been fully classified. MatrixNet sidesteps this by *learning* a matrix representation jointly with the downstream task, letting the representation adapt while still being constrained by the group's algebra.

5.2 Jordan–Hölder Multiplicities: The Validation Task

MatrixNet's main benchmark is predicting *Jordan–Hölder multiplicities* from the braid group's action on a category $\mathcal{C}_n$ via twist functors [1]. When a braid word w acts on an object $X \in \mathcal{C}_n$, the result has a composition series with irreducible multiplicities $\mathbf{m}(w) \in \mathbb{Z}^n_{\geq 0}$. These multiplicities depend only on the braid *element*, not the particular word, so any model that predicts them correctly must treat equivalent words the same. This makes the task a direct test of whether MatrixNet has learned the right equivalence structure.

5.3 Architecture

Each generator $\sigma_i^{\varepsilon_i}$ is encoded as a *signed one-hot* vector $v_{\sigma_i^{\varepsilon_i}} \in \mathbb{R}^n$ (with ε_i in position i, zeros elsewhere), so $v_{\sigma_i^{-1}} = -v_{\sigma_i}$. The *matrix block* maps this to an invertible matrix:

$$A_k = \mathrm{Reshape}(W v_{g_k}), \qquad M_{g_k} = \exp(A_k), \tag{4}$$

where $W \in \mathbb{R}^{d^2 \times n}$ is learnable. The matrix exponential guarantees $M_{g_k} \in \mathrm{GL}(d, \mathbb{R})$ and automatically gives $M_{g^{-1}} = M_g^{-1}$. For a word $w = g_{i_1} \cdots g_{i_\ell}$:

$$M(w) = M_{g_{i_1}} \cdots M_{g_{i_\ell}}, \qquad s(w) = \mathrm{flatten}(M(w)) \in \mathbb{R}^{d^2}. \tag{5}$$

This defines a homomorphism from F_S to $\mathrm{GL}(d, \mathbb{R})$ (Proposition 1 of [1]).

5.4 The Relation Loss

The matrix block by itself only gives a free-group representation—there's no reason for the learned matrices to satisfy the braid relations. The *relation loss* fixes this:

$$\mathcal{L}_{\mathrm{rel}} = \sum_{r_i \in R} \|M(r_i) - I_d\|_F^2 \,, \tag{6}$$

summing over all defining relations of B_n. The total loss is $\mathcal{L}_{\mathrm{total}} = \mathcal{L}_{\mathrm{task}} + \lambda\, \mathcal{L}_{\mathrm{rel}}$, where λ balances task accuracy against algebraic faithfulness. When $\mathcal{L}_{\mathrm{rel}}$ is small, equivalent words map to nearly identical matrices, and the model can extrapolate to words *longer* than any in training. Baselines without this inductive bias can't do that.

MatrixNet has four variants (baseline, two-layer linear LN, nonlinear NL with tanh, and multi-channel block-diagonal MC); see [1] for details.

5.5 Empirical Results on B_3

Table 1: MSE and exact-match accuracy for Jordan–Hölder multiplicity prediction on B_3. All models have roughly the same parameter count. Results averaged over 5 runs (Laird et al., 2025).

Model	**Parameters**	**MSE (epoch 100)**	**Accuracy**
Transformer	63,779	2.895 ± 0.024	42.3%
MLP	52,099	0.132 ± 0.009	89.1%
LSTM	51,027	0.075 ± 0.035	93.0%
MatrixNet	42,507	0.082 ± 0.034	95.1%
MatrixNet-LN	42,883	0.001 ± 0.001	**99.9**%
MatrixNet-MC	41,987	0.014 ± 0.006	98.8%
MatrixNet-NL	42,883	$6.4 \times 10^{-4} \pm 3.6 \times 10^{-4}$	**99.9**%

The results here are pretty striking. The Transformer (42.3%, MSE ≈ 2.9) basically doesn't work—even though it's the default architecture for sequences, it has no algebraic inductive bias and can't learn to identify equivalent words from the data alone. The MLP (89.1%) and LSTM (93.0%) do okay but struggle to generalize beyond training words. MatrixNet-LN and MatrixNet-NL both hit 99.9% with orders-of-magnitude lower MSE, using *fewer* parameters. The key differentiator is how well $\mathcal{L}_{\text{rel}}$ gets minimized: models with near-zero relational error extrapolate to longer words; those without it don't. Enforcing algebraic structure matters way more than just adding parameters.

With the symmetry-aware representation in hand, Section 6 puts together the full RL pipeline.

6 Reinforcement Learning via MatrixNet on the Braid Group

6.1 Assembling the Framework

Now we bring all three pieces together. The goal: given an arbitrary braid word $w \in B_n$, find its Garside normal form $\Delta_n^p A_1 \cdots A_\ell$ (Theorem 4.4) through a learned sequence of algebraic moves. Figure 1 shows the full pipeline.

6.2 The State Representation

Instead of handing the raw word w to the RL agent—which would force it to separately learn behavior for every word representing the same braid—we use the MatrixNet embedding:

$$s(w) = \text{flatten}(M(w)) = \text{flatten}(M_{g_{i_1}} \cdots M_{g_{i_\ell}}) \in \mathbb{R}^{d^2}. \tag{7}$$

Since $M(w_1) \approx M(w_2)$ whenever $w_1 \equiv w_2$ in B_n (thanks to the relation loss), equivalent words land in the same region of state space. The agent effectively operates on a compressed space where each braid element has one representative embedding.

6.3 The Action Space

The agent can apply three types of actions:

1. **Generator multiplication:** $w \mapsto \sigma_i^{\pm 1} \cdot w$ or $w \mapsto w \cdot \sigma_i^{\pm 1}$, for $i \in \{1, \ldots, n-1\}$. This gives $4(n-1)$ actions.
2. **Relation rewriting:** Replace a subword matching one side of a braid relation with the other side (e.g., swap $\sigma_i\sigma_{i+1}\sigma_i$ for $\sigma_{i+1}\sigma_i\sigma_{i+1}$). These don't change the braid element but can move the word closer to normal form.
3. **Cancellation:** Remove adjacent inverse pairs $\sigma_i\sigma_i^{-1}$ or $\sigma_i^{-1}\sigma_i$, reducing word length.

6.4 The Reward Function

Each step gives a composite reward:

$$r_t = \underbrace{(|w| - |w'|)}_{r_{\text{simp}}} + \beta \underbrace{\left(-\sum_{r_i \in R} \|M(r_i) - I\|_F^2 \right)}_{r_{\text{rel}}} + \kappa\, r_{\text{canon}} - \delta. \tag{8}$$

Here r_{simp} rewards word-length reduction, r_{rel} penalizes algebraic inconsistencies, $r_{\text{canon}} = C$ is a big terminal bonus for reaching Garside normal form, and δ is a small per-step cost to discourage wandering.

6.5 Policy Learning

The policy $\pi_\theta(a \mid s(w))$ and value network $V_\theta(s(w))$ take the MatrixNet embedding as input. Because $s(w_1) \approx s(w_2)$ for equivalent words, the policy is automatically symmetry-equivariant. The classical Garside algorithm corresponds to one particular deterministic policy (always

apply the left-greedy operator); the RL approach could potentially find more efficient paths for specific braid families.

7 Further Applications

The braid group / MatrixNet / RL combination has potential beyond just Garside normalization. **Knot simplification:** Reidemeister moves form an analogous RL action space; MatrixNet could encode diagram states while respecting Reidemeister equivalence. **Topological robotics:** Ropes, cables, and sutures have configuration spaces that decompose into braid equivalence classes, so an RL agent with MatrixNet representations could plan at the level of topological subgoals. **Multi-robot coordination:** The trajectories of n planar robots form a braid, and cable entanglement between them is literally a braid group problem. **Topological quantum computation:** Non-Abelian anyons implement quantum gates via braiding [9]; circuit synthesis is a search over braid words for target unitaries, and our RL framework applies directly.

8 Proposed Extension: Predicting the Plamenevskaya Transverse Invariant

Everything up to this point has been about using MatrixNet to guide RL toward a canonical form. In this section, we switch gears and outline a second application of MatrixNet's learned representations that we think is promising but *haven't implemented yet*: predicting properties of *Khovanov homology* directly from braid words. We lay out the mathematical background, explain why MatrixNet's architecture seems like a good fit, and describe concretely how someone could set up this experiment.

8.1 Contact Geometry and Transverse Knots

A *contact structure* on a 3-manifold Y is a maximally non-integrable plane field $\xi \subset TY$. The standard example on $\mathbb{R}^3$ is $\xi_{\text{std}} = \ker(dz - y\,dx)$. A knot $K \subset (Y, \xi)$ is *transverse* if its tangent vector is everywhere transverse to ξ.

What makes transverse knots interesting is that two of them can be the same topological knot but *different* transverse knots—you can't deform one into the other while staying transverse. So you need invariants that see the contact structure, not just the topology.

The connection to braids is direct: the closure $\hat{\beta}$ of any braid $\beta \in B_n$ naturally inherits a

transverse structure, and Bennequin (1983) showed every transverse knot arises this way. So braid algebra ends up encoding contact geometry.

8.2 Khovanov Homology

Khovanov homology [14] categorifies the Jones polynomial. Given a link diagram D with k crossings, you resolve each crossing in two ways, getting 2^k complete resolutions. Each resolution gets a graded vector space (from the Frobenius algebra $\mathbb{Z}[x]/(x^2)$), and these assemble into a bi-graded chain complex $\mathrm{CKh}^{i,j}(D)$ whose homology $\mathrm{Kh}(D)$ is a link invariant. It's strictly stronger than the Jones polynomial and detects the unknot (Kronheimer–Mrowka, 2011).

The catch: the chain complex has $O(2^k)$ generators, and computing the homology is #P-hard. Even moderate crossing numbers make it intractable—which is exactly why a learned shortcut would be valuable.

8.3 The Plamenevskaya Invariant $\psi(\beta)$

Plamenevskaya (2006) identified a special element inside the Khovanov chain complex of a braid closure that carries contact-geometric information [13].

Definition 8.1 (Plamenevskaya, 2006)**.** Let $\beta \in B_n$ be a braid word. The *all-0 resolution* of the braid closure $\hat{\beta}$ resolves every crossing with a 0-smoothing, producing n disjoint simple closed curves. Assign $x \in \mathbb{Z}[x]/(x^2)$ to each curve. The tensor product $x^{\otimes n}$ defines an element $\psi(\beta) \in \mathrm{CKh}^{0,j_0}(\hat{\beta})$ at homological grading 0 and a quantum grading j_0 determined by the writhe of β.

The important property: $\psi(\beta)$, as a class in $\mathrm{Kh}(\hat{\beta})$, is a *transverse knot invariant*. If two braids have transversely isotopic closures, then ψ vanishes for one iff it vanishes for the other.

8.3.1 ψ as a Quasipositivity Obstruction

A braid $\beta \in B_n$ is *quasipositive* if it can be written as $\beta = \prod_j w_j \sigma_{i_j} w_j^{-1}$. Quasipositive braids are geometrically natural—their closures are exactly the links arising as transverse intersections of algebraic curves with S^3 (Rudolph, 1983).

Plamenevskaya showed: if β is quasipositive, then $\psi(\beta) \neq 0$. Contrapositively:

$$\psi(\beta) = 0 \quad \Longrightarrow \quad \beta \text{ is not quasipositive.} \tag{9}$$

So ψ vanishing gives a certificate of non-quasipositivity that goes beyond what classical

numerical invariants can detect.

8.4 Proposed Task: Binary Classification of $\psi(\beta)$

The idea would be to set this up as a binary classification problem:

$$f : B_n \to \{0, 1\}, \qquad f(\beta) = \begin{cases} 1 & \text{if } \psi(\beta) = 0, \\ 0 & \text{if } \psi(\beta) \neq 0. \end{cases} \tag{10}$$

Given a braid word β, a trained model would predict $f(\beta)$ directly from the generator sequence, skipping the #P-hard chain complex computation entirely. One forward pass would replace an exponential-time calculation.

This is well-defined because $\psi(\beta)$ depends on the braid element $[\beta] \in B_n$ (up to positive stabilization), not the word. So any correct model would need to give equivalent words the same prediction—exactly the invariance MatrixNet is built to enforce.

8.5 Why MatrixNet's Architecture Fits This Problem

We haven't run these experiments, but there are three structural reasons to think MatrixNet would work well here:

(1) Sign structure alignment. The all-0 resolution defining ψ depends critically on whether each crossing is positive or negative—positive crossings get horizontal smoothings, negative ones get vertical. MatrixNet's signed one-hot encoding ($v_{\sigma_i^{-1}} = -v_{\sigma_i}$) mirrors this directly, and the matrix exponential maps it to $M_{\sigma_i^{-1}} = M_{\sigma_i}^{-1}$. The architecture bakes in the same sign distinction that determines the topology of the resolution.

(2) Continuous, invertible parametrization. The matrix exponential $M_{\sigma_i} = \exp(A_i)$ puts the discrete data of generators into a differentiable manifold of invertible matrices. Whether ψ vanishes is binary, but the algebraic features determining the answer likely vary smoothly across braid families. The matrix exponential would let the model learn smooth decision boundaries—something a discrete encoding couldn't do as easily.

(3) Relational invariance. Since $\psi(\beta)$ is a transverse invariant, equivalent braid words must get the same classification. The relation loss (Equation 6) ensures $M(w_1) \approx M(w_2)$ when $w_1 \equiv w_2$, which would automatically handle this. Positive-stabilization invariance isn't

directly enforced but could in principle be learned from data, and the relation loss provides a strong inductive bias toward respecting algebraic equivalences in general.

8.6 How We Would Set This Up

8.6.1 Dataset Construction

The training data would come from enumerating braid words in B_3 and B_4 up to a given length and computing $\psi(\beta)$ for each by brute-force construction of the Khovanov chain complex. For B_3, words up to length ~ 12 should be tractable; for B_4, probably lengths around 8–10 depending on the specific braid. You'd label each word with $f(\beta) \in \{0, 1\}$ and balance the classes by oversampling whichever is rarer (we expect $\psi = 0$ to be the minority for short words).

The key test would be to hold out braid words *longer* than anything in training. If the model can extrapolate to unseen lengths, that's strong evidence it learned real algebraic structure rather than just memorizing patterns at specific word lengths. This is the same kind of length-extrapolation test that validated MatrixNet on the Jordan–Hölder task (Section 5.5).

8.6.2 Loss Function

The total loss would combine binary cross-entropy for the classification with the same relation loss from Equation (6):

$$\mathcal{L}_\psi = \underbrace{-\frac{1}{N}\sum_{k=1}^{N}\Big[y_k \log \hat{f}(\beta_k) + (1-y_k)\log(1-\hat{f}(\beta_k))\Big]}_{\mathcal{L}_{\mathrm{BCE}}} + \lambda\,\mathcal{L}_{\mathrm{rel}}. \tag{11}$$

The relation loss is identical to the Jordan–Hölder setting, and λ would again control the accuracy-vs-faithfulness trade-off. Based on the Jordan–Hölder results, we'd expect that models with low $\mathcal{L}_{\mathrm{rel}}$ would generalize better to longer words, while baselines without the relation loss would struggle to extrapolate.

8.6.3 What Success Would Look Like

If MatrixNet achieves high accuracy on $\psi(\beta)$ classification—especially on words longer than those in training—that would mean two things. First, it would confirm that the learned representations capture genuine topological information, not just word-level statistics. Second, it would give us a fast proxy for a #P-hard computation, which could be useful for exploring conjectures in contact topology (e.g., scanning large families of braids for non-quasipositive

examples). On the other hand, if the model fails to extrapolate, that would suggest that the vanishing of ψ depends on algebraic features that MatrixNet's particular parametrization can't capture, which would itself be an interesting finding.

9 Conclusion

We've put together a framework for solving the braid word problem using RL guided by learned matrix representations. The three ingredients—RL for sequential optimization, the braid group B_n with its Garside normal form as the target, and MatrixNet for symmetry-aware state encoding—fit together naturally. MatrixNet's performance on the Jordan–Hölder task (reported by Laird et al.) confirms it can learn braid equivalence, and our pipeline (Figure 1) shows how this feeds into an RL agent.

We also outlined a proposed application of MatrixNet to predicting $\psi(\beta)$ vanishing in Khovanov homology (which may or may not be implemented depending on how generous the people at URAD are feeling). The architectural alignment between MatrixNet and the structure of ψ—the sign encoding, the continuous parametrization, the relational invariance—makes us think this is a promising direction. If it works, it would give a fast learned proxy for a #P-hard computation and open the door to ML-assisted exploration in contact topology.

What makes the RL framework work (in principle). Three design choices are doing the heavy lifting:

(1) The *matrix exponential* guarantees invertibility, automatically enforcing inverse and identity axioms.

(2) The *relation loss* teaches the representation to identify equivalent words.

(3) The *sequential multiplication* structure mirrors word structure in the braid group, enabling length generalization.

Open questions and next steps. The most immediate next step is actually implementing and testing the $\psi(\beta)$ classification pipeline described in Section 8. Beyond that, scaling to larger braid groups ($n > 3$) means dealing with exponentially growing action and state spaces. Richer reward signals—like Thurston's automatic structure [8]—might help RL agents find new normalization strategies. Understanding what MatrixNet's learned representations look like in terms of known irreducible representations of B_n is an open theoretical question. And on the topological side, extending from $\psi(\beta)$ to other Khovanov-theoretic invariants (like Rasmussen's s-invariant [15]) could test whether MatrixNet learns a unified algebraic

encoding of homological information or whether different invariants need fundamentally different representations.

References

[1] L. Laird, C. Hsu, A. Bapat, and R. Walters, "MatrixNet: Learning over symmetry groups using learned group representations," in *Proceedings of NeurIPS 2024*, 2025. arXiv:2501.09571.

[2] E. Artin, "Theorie der Zöpfe," *Abh. Math. Sem. Univ. Hamburg*, vol. 4, pp. 47–72, 1925.

[3] F. A. Garside, "The braid group and other groups," *Q. J. Math.*, vol. 20, pp. 235–254, 1969.

[4] J. W. Alexander, "A lemma on systems of knotted curves," *Proc. Natl. Acad. Sci.*, vol. 9, pp. 93–95, 1923.

[5] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.

[6] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[7] V. F. R. Jones, "A polynomial invariant for knots via von Neumann algebras," *Bull. Amer. Math. Soc.*, vol. 12, pp. 103–111, 1985.

[8] W. Thurston, "Finite state algorithms for the braid groups," Unpublished manuscript, 1992.

[9] C. Nayak, S. H. Simon, A. Stern, M. Freedman, and S. Das Sarma, "Non-Abelian anyons and topological quantum computation," *Rev. Mod. Phys.*, vol. 80, pp. 1083–1159, 2008.

[10] A. Davies et al., "Advancing mathematics by guiding human intuition with AI," *Nature*, vol. 600, pp. 70–74, 2021.

[11] J. S. Birman, *Braids, Links, and Mapping Class Groups*, Princeton Univ. Press, 1974.

[12] P. Dehornoy, "Efficient solutions to the braid isotopy problem," *Discr. Appl. Math.*, vol. 156, pp. 3094–3112, 2008.

[13] O. Plamenevskaya, "Transverse knots and Khovanov homology," *Math. Res. Lett.*, vol. 13, no. 4, pp. 571–586, 2006.

[14] M. Khovanov, "A categorification of the Jones polynomial," *Duke Math. J.*, vol. 101, no. 3, pp. 359–426, 2000.

[15] J. Rasmussen, "Khovanov homology and the slice genus," *Invent. Math.*, vol. 182, no. 2, pp. 419–447, 2010.